

Architettura degli Elaboratori

Parallelismo



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

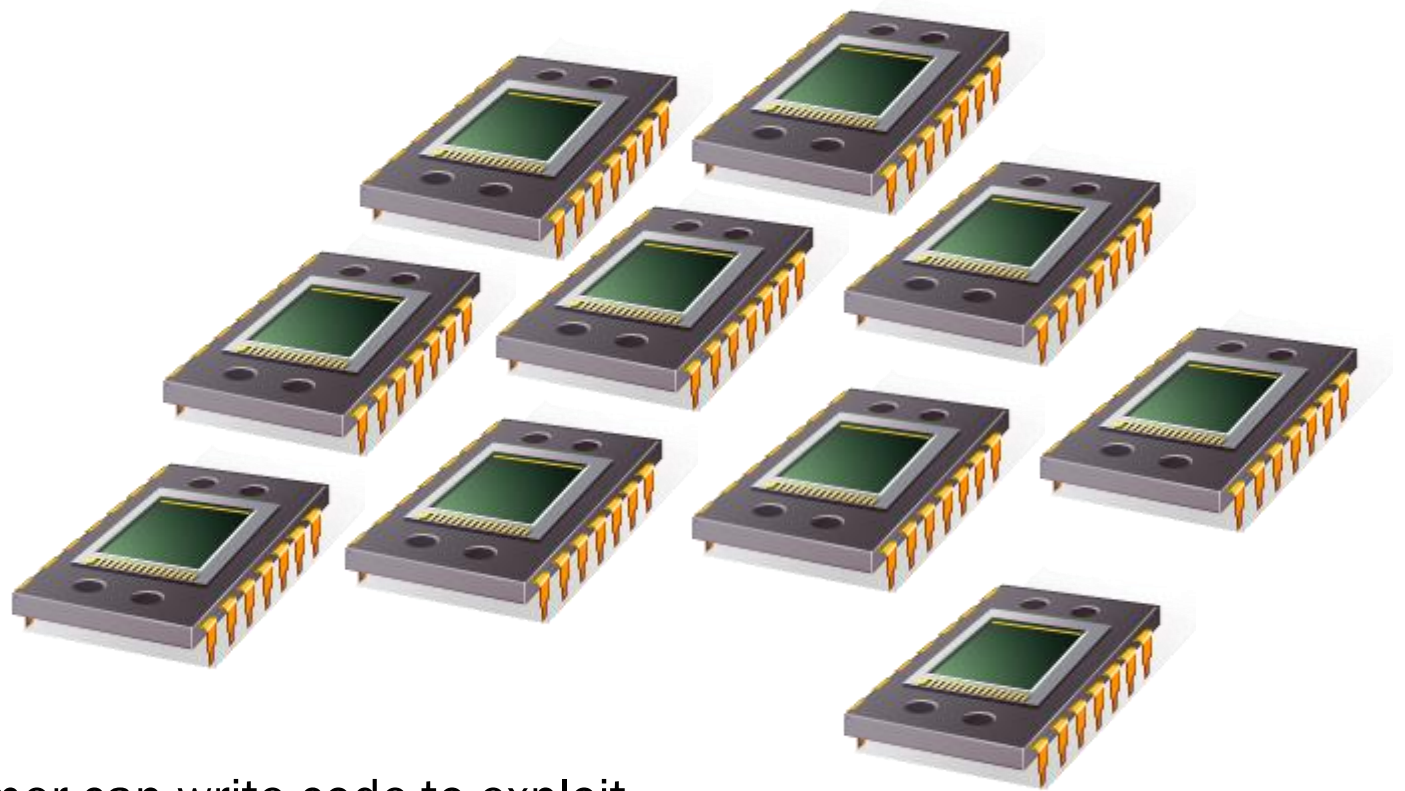
davide.polese@uniroma1.it

Special thanks and credits:
Salvatore Pontarelli

[S&PdC]

6





A programmer can write code to exploit.

PARALLEL HARDWARE

Introduzione

- **Goal: Aumentare le performance usando più processori**
 - Multiprocessori
 - Scalabilità, disponibilità, efficienza energetica
- Parallelismo a livello di task e/o di processo
 - Grande throughput per task/processi indipendenti
- Esecuzione parallelo programmi
 - Un singolo programma viene eseguito su più processori
- Processori multicore
 - Chips con CPU multiple (cores)
- Clusters
 - Calcolatori indipendenti connessi tramite una rete locale

Threads e processi

- **Thread:** Un thread richiede un program counter, un registro di stato e lo stack
 - Più thread possono condividere lo stesso spazio di indirizzamento.
- **Processo:** un processo comprende uno o più threads, lo spazio di indirizzamento e lo stato del OS
- Il cambio di Processo richiede più tempo del cambio di thread.

Hardware and Software

Hardware

Seriale: es. Pentium 4

Parallelo: es. quad-core Xeon e5345

Software

Sequenziale: es. Moltiplicazioni matriciali

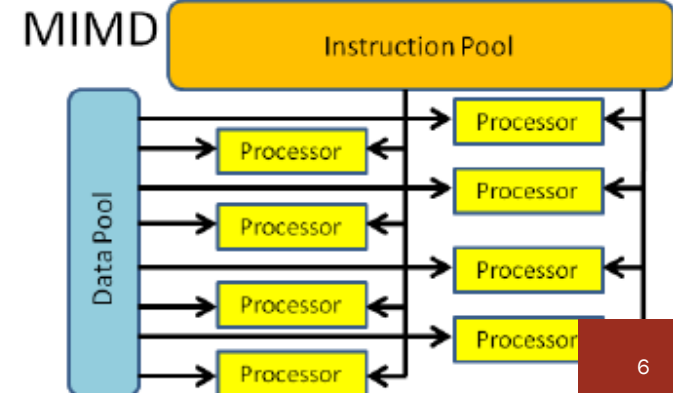
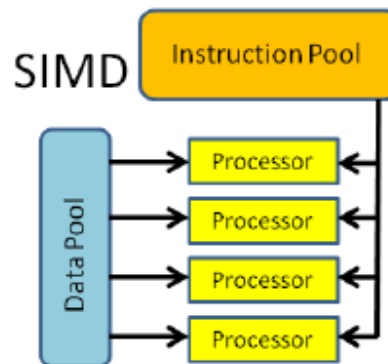
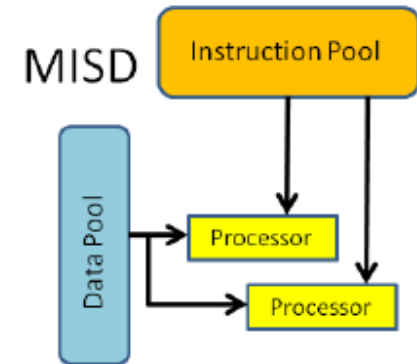
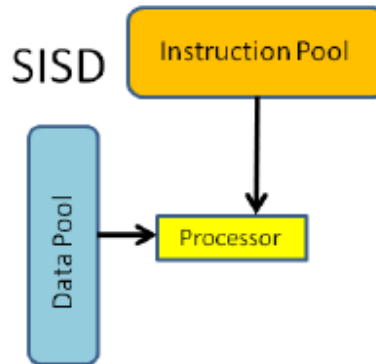
Concorrente: es. Sistema operativo

Il software sequenziale/concorrente può essere eseguito su hardware seriale/parallelo
Challenge: rendere efficace l'utilizzo di hardware parallelo

		Software	
		Sequenziale	Concorrente
Hardware	Seriale	Moltiplicazione di matrici in MATLAB in esecuzione su un Pentium 4 Intel	Sistema operativo Windows Vista in esecuzione su un Pentium 4 Intel
	Parallelo	Moltiplicazione di matrici in MATLAB in esecuzione su un Core i7 Intel	Sistema operativo Windows Vista in esecuzione su un Core i7 Intel

Tassonomia di Flynn

		Data Streams	
		Single	Multiple
Instruction Streams	Single	SISD: Intel Pentium 4	SIMD: SSE instructions of x86
	Multiple	MISD: No examples today	MIMD: Intel Xeon e5345



Programmazione Parallela

Scrivere programmi paralleli è difficile:

- Bisogna ottenere un significativo miglioramento delle prestazioni altrimenti è meglio utilizzare un processore single core più veloce.
- Difficoltà:
 - Partizionamento
 - Coordinamento
 - Costo di comunicazione aggiuntivo

Legge di Amdahl

La parte sequenziale può limitare l'aumento della velocità:

$$\text{Speedup} = \frac{T_{old}}{T_{new}} = \frac{(T_{parallelizabile} + T_{sequenziale})}{T_{new}}$$

$$F_{parallelizabile} = \frac{T_{parallelizabile}}{(T_{parallelizabile} + T_{sequenziale})}$$

Esempio: N processori, speedup?

$$T_{new} = \frac{T_{parallelizzabile}}{N} + T_{sequenziale}$$

$$\text{Speedup} = \frac{1}{(1 - F_{parallelizabile}) + \frac{F_{parallelizabile}}{N}}$$

Legge di Amdahl

Esempio: 100 processori, 90× speedup

$$Speedup = \frac{1}{(1 - F_{parallelizabile}) + \frac{F_{parallelizabile}}{N}}$$

$$F_{parallelizabile} = \frac{N}{1 - N} \cdot \frac{1 - Speedup}{Speedup} = \frac{100}{1 - 100} \cdot \frac{1 - 90}{90} = 0.999$$

La parte sequenziale deve essere minore dello 0.1% del tempo originale!

Scaling Example

Carico: somma di 10 scalari e due matrici 10x10

Speed up da 10 a 100 processori

Processore singolo: tempo = $(10 + 100) \times t_{\text{add}}$

10 processori

$$\text{Tempo} = 10 \times t_{\text{add}} + 100/10 \times t_{\text{add}} = 20 \times t_{\text{add}}$$

$$\text{Speedup} = 110/20 = 5.5 \text{ (55\% of potential)}$$

100 processori

$$\text{Time} = 10 \times t_{\text{add}} + 100/100 \times t_{\text{add}} = 11 \times t_{\text{add}}$$

$$\text{Speedup} = 110/11 = 10 \text{ (10\% of potential)}$$

Si assume che il carico possa essere bilanciato tra tutti i processori

Scaling Example (cont)

Cosa succederebbe se la matrice fosse 100x100?

Processore singolo: Tempo = $(10 + 10000) \times t_{add}$

10 processori

$$\text{Tempo} = 10 \times t_{add} + 10000/10 \times t_{add} = 1010 \times t_{add}$$

$$\text{Speedup} = 10010/1010 = 9.9 \text{ (99\% of potenziale)}$$

100 processori

$$\text{Time} = 10 \times t_{add} + 10000/100 \times t_{add} = 110 \times t_{add}$$

$$\text{Speedup} = 10010/110 = 91 \text{ (91\% of potenziale)}$$

Assumendo che il carico sia bilanciato

Scalabilità debole e forte

- **Scalabilità forte:** Misura l'incremento di velocità di esecuzione quando si mantiene fissa la dimensione del problema
- **Scalabilità debole:** Misura l'incremento di velocità di esecuzione quando la dimensione del problema cresce proporzionalmente al numero dei processori

Weak scaling: problem size proportional to number of processors

10 processors, **10 × 10** matrix

$$\text{Time} = 20 \times t_{\text{add}}$$

100 processors, **32 × 32** matrix

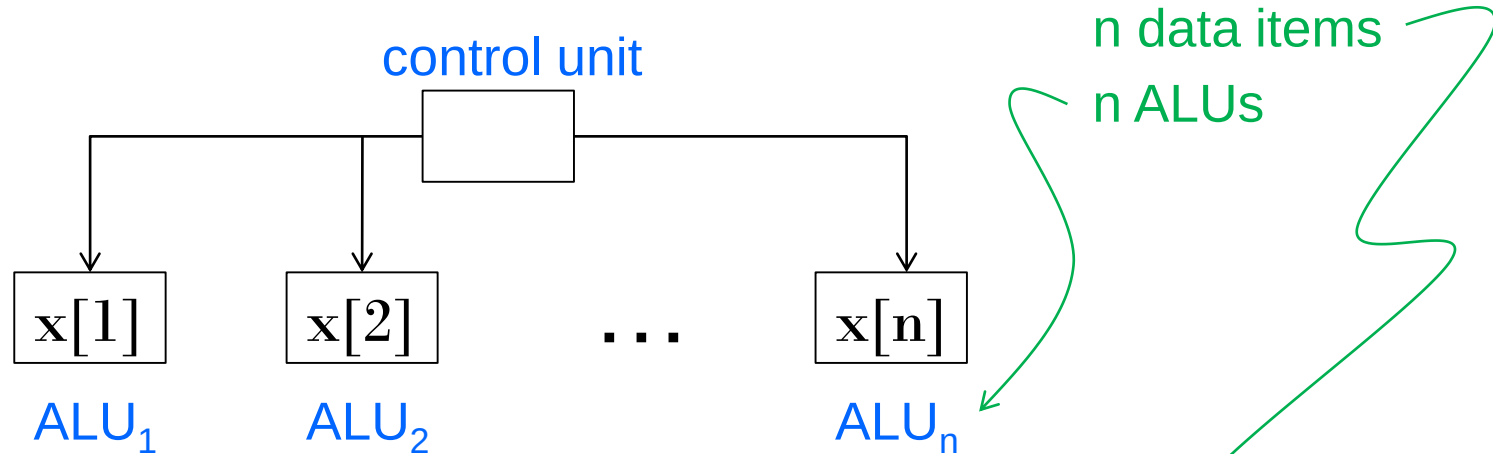
$$\text{Time} = 10 \times t_{\text{add}} + 1000/100 \times t_{\text{add}} = 20 \times t_{\text{add}}$$

Constant performance in this example

Diverse forme di parallelismo

- ILP Parallelismo a livello di istruzione: Pipeline
- DLP Parallelismo a livello di dati: SIMD, Architetture vettoriali
 - es. istruzioni MMX, SSE, SSE2, AVX in architetture x86

SIMD example



```
for (i = 0; i < n; i++)  
    x[i] += y[i];
```

Il parallelismo si ottiene dividendo i dati tra più processori
Applica la stessa istruzione a più dati.

Parallelismo sui dati

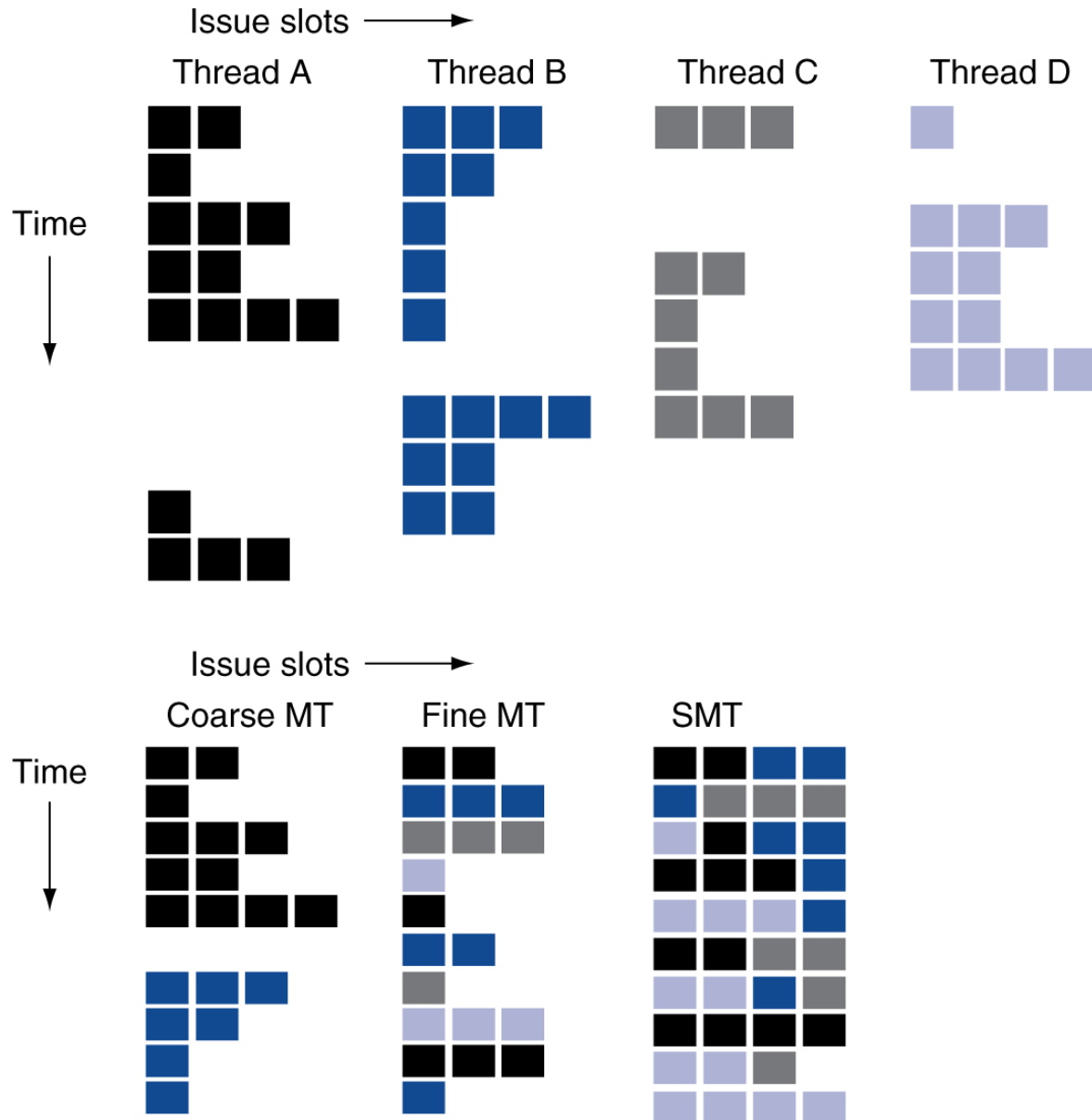
Svantaggi delle architetture SIMD

- Tutte le ALU devono eseguire al stessa istruzione o rimanere ferme
- In architetture classiche devono operare in modo sincrono
- Le ALU non hanno buffer delle istruzioni
- E' efficiente per problemi paralleli di grande dimensione, ma non per altri problemi paralleli complessi

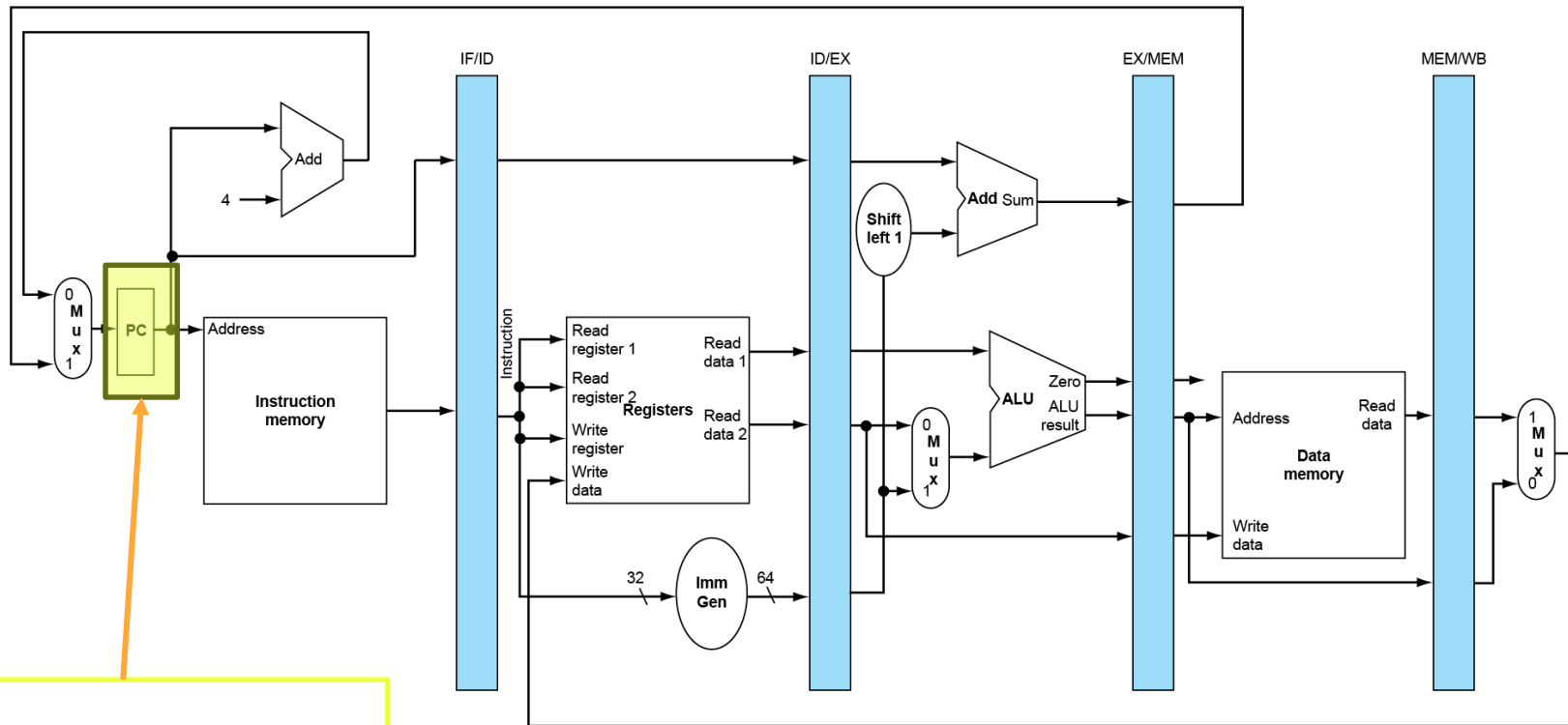
Multithreading

- **Processare diversi thread di esecuzione in parallelo**
 - Replicare i registri, PC, etc.
 - Cambio veloce di thread
- **Multithreading a grana fine**
 - Cambio di thread ad ogni ciclo
 - Esecuzione intrecciata dei threads
 - Se un thread è in stallo, si eseguono gli altri thread
 - Rallenta l'esecuzione del singolo thread
- **Multithreading a grana grossa**
 - Il cambio di thread viene effettuato solo per stalli lunghi (es., L2-cache miss)
 - Semplifica l'hardware, ma non maschera gli stalli corti (es., data hazards)
 - Ha alto costo di cambio thread (es. riavvio della pipeline)
- **Multithreading simultaneo (SMT)**
 - L'hardware sceglie quali istruzioni eseguire dei vari thread
 - Le istruzioni da thread indipendenti sono eseguite quando le unità funzionali sono disponibili
 - All'interno del thread, le dipendenze sono gestite dall'hardware
 - Example: Intel Pentium-4 HT Two threads: duplicated registers, shared function units and caches

Multithreading Example

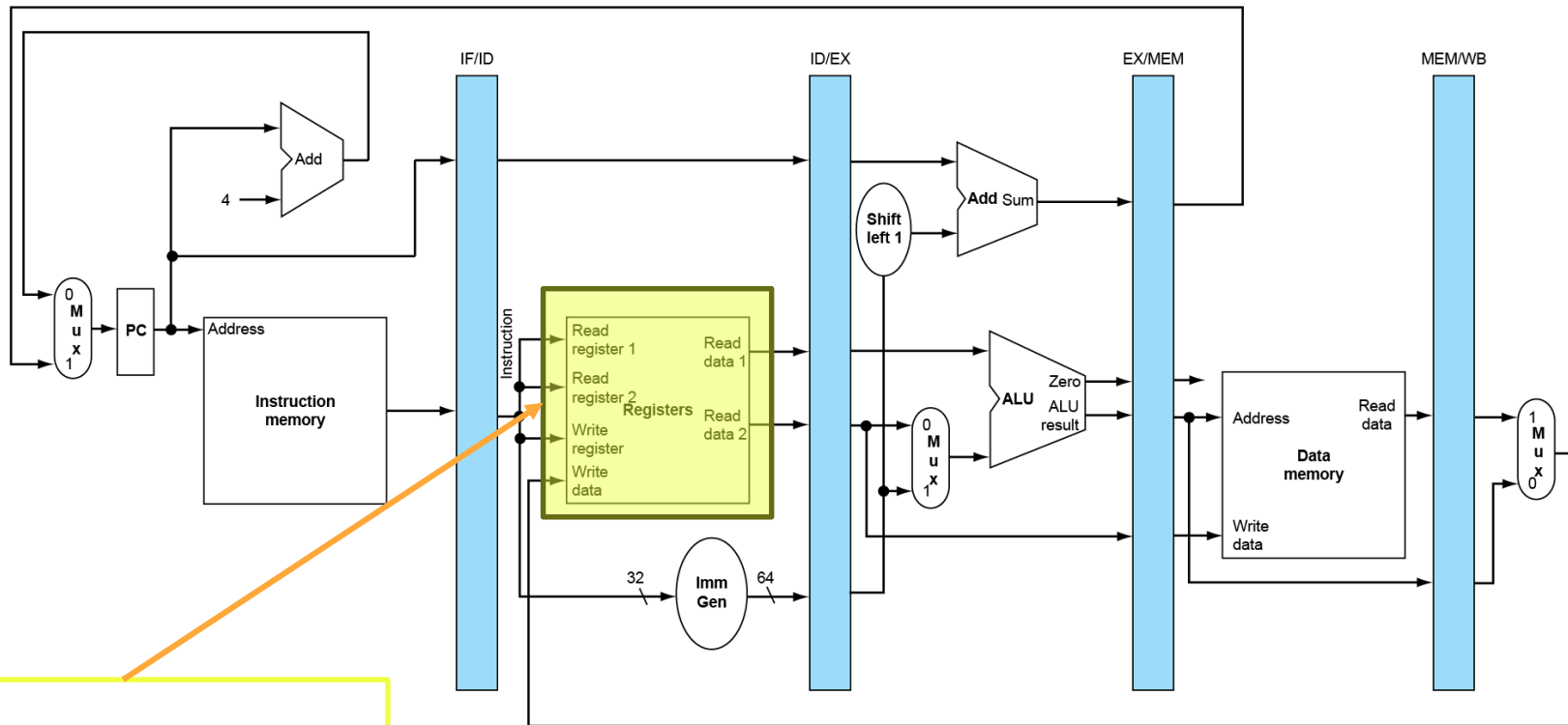


Fine grain multithreading



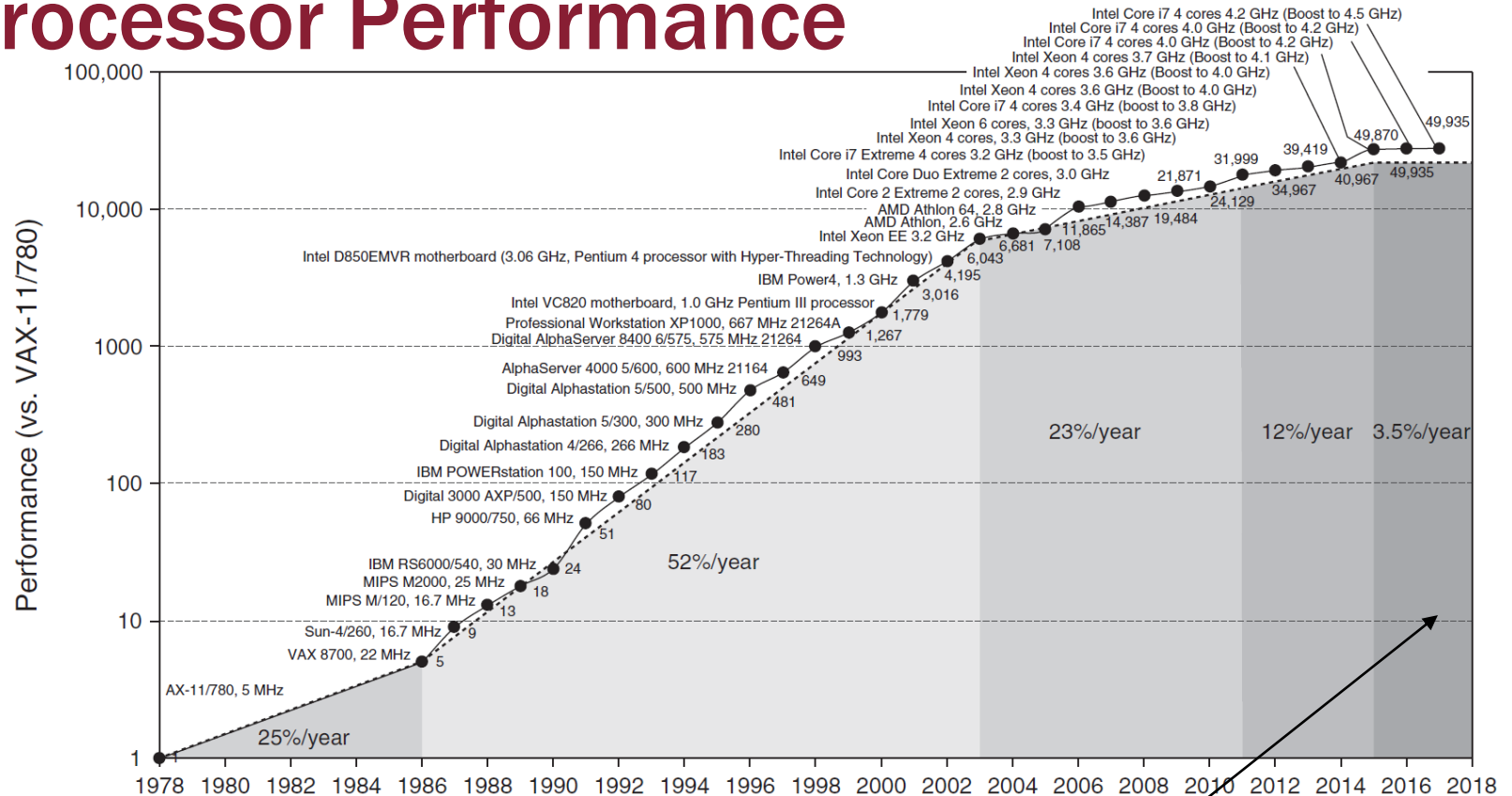
Use multiple PC registers

Fine grain multithreading



And multiple register files

Uniprocessor Performance



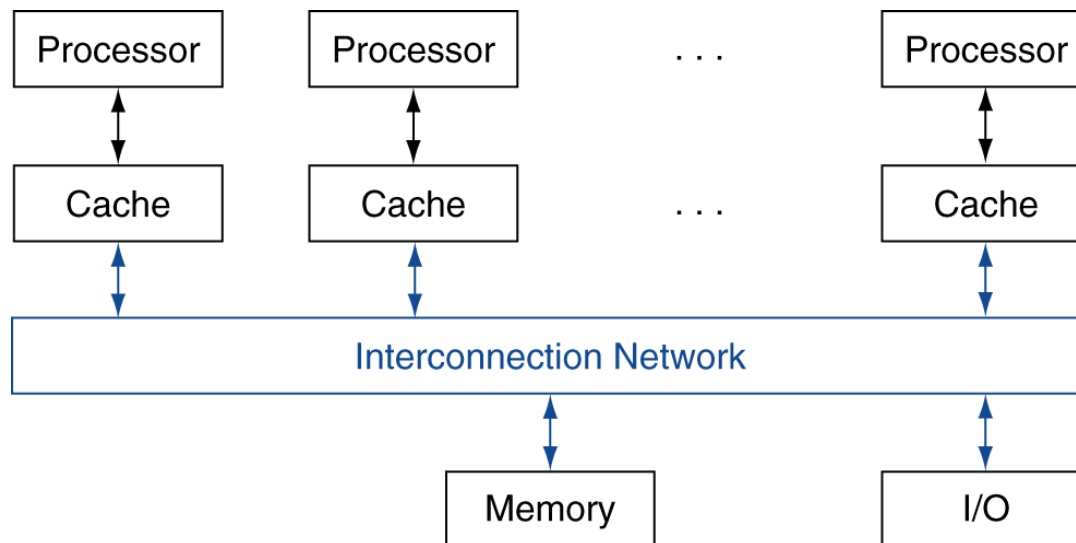
Constrained by power, instruction-level parallelism, memory latency

Processori Multicore

- Processori multicores: più processori (core) per chip
- Permette di superare il limite di potenza dovuto all'incremento della frequenza di clock
- Ha un impatto significativo sul software

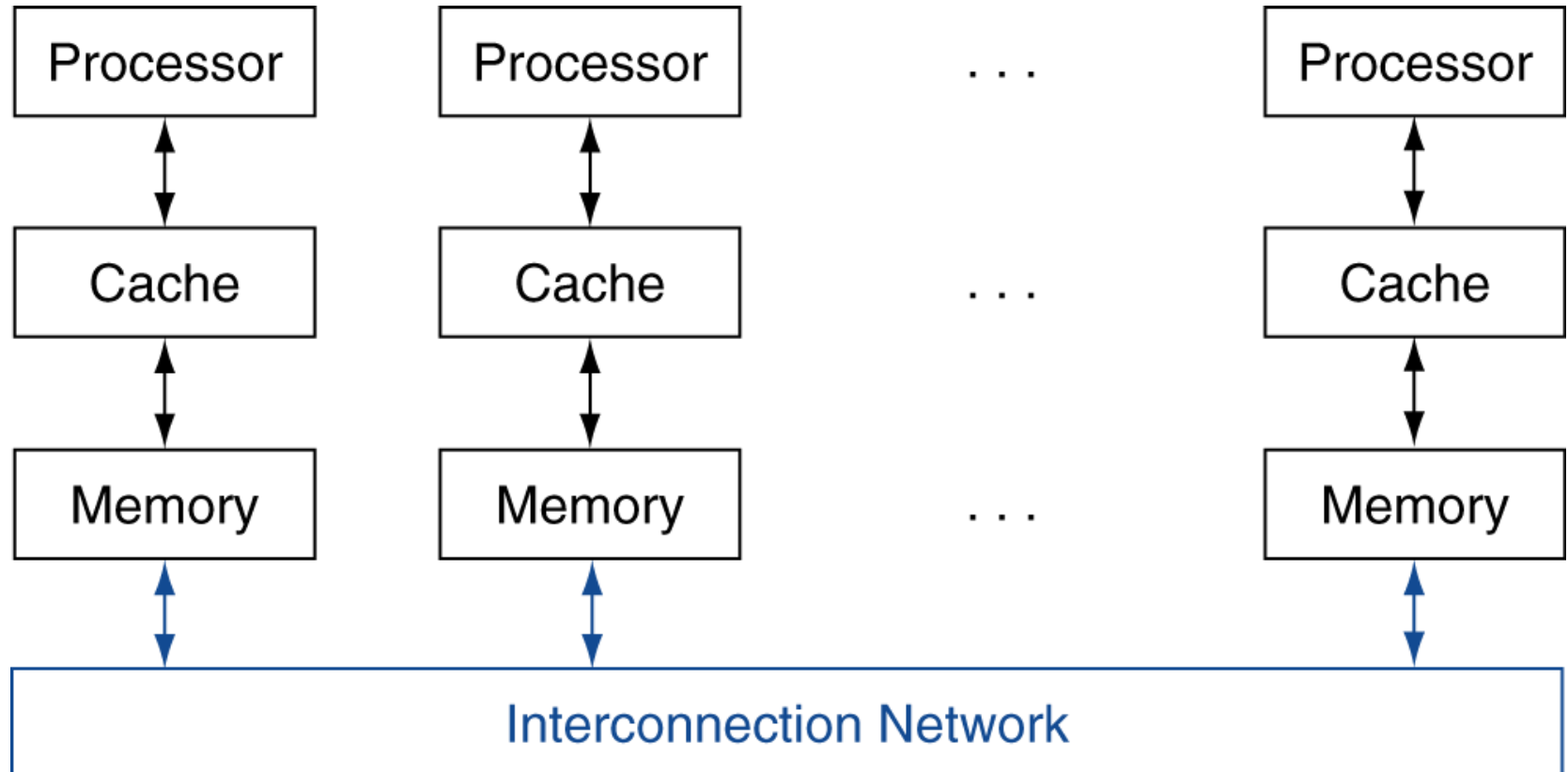
Processori a memoria condivisa

- **SMP: shared memory multiprocessor**
 - L'hardware fornisce un unico spazio di indirizzamento fisico per tutti i cores (i programmi non sanno quale processore li eseguirà ma solo che sono eseguiti in parallelo)
 - Sincronizzazione delle variabile usando «locks»
 - Il tempo di accesso alla memoria principale
 - **UMA (uniform):** il tempo di accesso a qualsiasi locazione di memoria indipendentemente dal core che ha richiesto la locazione
 - **NUMA (nonuniform):** Il tempo di accesso ad una locazione di memoria dipende dal dato richiesto e dal processore che ha richiesto il dato



Message Passing

- Ogni processore ha uno spazio di indirizzamento fisico proprio
- L'hardware invia e riceve messaggi dai vari cores



Esempio: Somma con riduzione

- Somma di 64,000 numeri su 64 processori UMA

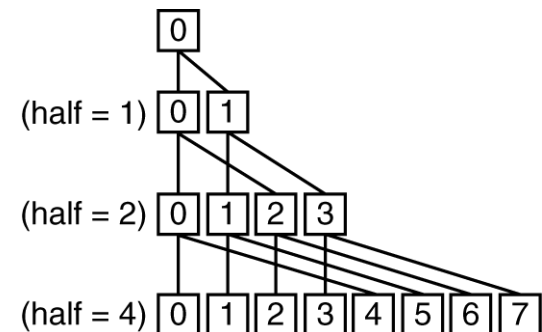
- Ogni processore ha un ID: $0 \leq P_n \leq 63$
- 1000 numeri per processore (Partizionamento)
- Ogni processore esegue il seguente codice:

```
sum[Pn] = 0;
for (i = 1000*Pn; i < 1000*(Pn+1); i += 1)
    sum[Pn] += A[i];
```

- C'è la necessità di sommare le somme parziali

- Riduzione: divide and conquer
- Metà dei processori sommano le coppie, poi un quarto, ...
- C'è la necessità di sincronizzare questi steps

```
half = 64;
do
    synch();
    half = half/2; /* dividing line on who sums */
    if (Pn < half) sum[Pn] += sum[Pn+half];
while (half > 1);
```



Sincronizzazione

Sincronizzazione: processo che coordina il comportamento di due o più threads, che possono essere eseguiti su differenti processori

- Un approccio usa con «lock» per le variabili condivise. Solo un processore alla volta può acquisire il «lock», e gli altri processori interessati nella variabile condivisa devono aspettare finché il primo processore non rilascia la variabile
- Richiede istruzioni hardware specifiche (atomiche) per evitare «race conditions»

Cache multiple per processori multipli

Problema:

- In un sistema multi-processore si usano cache multiple in parallelo
- Processi diversi possono contemporaneamente accedere/modificare gli stessi dati
- Bisogna mantenere «coerenti» le informazioni tra i diversi processori

Idea:

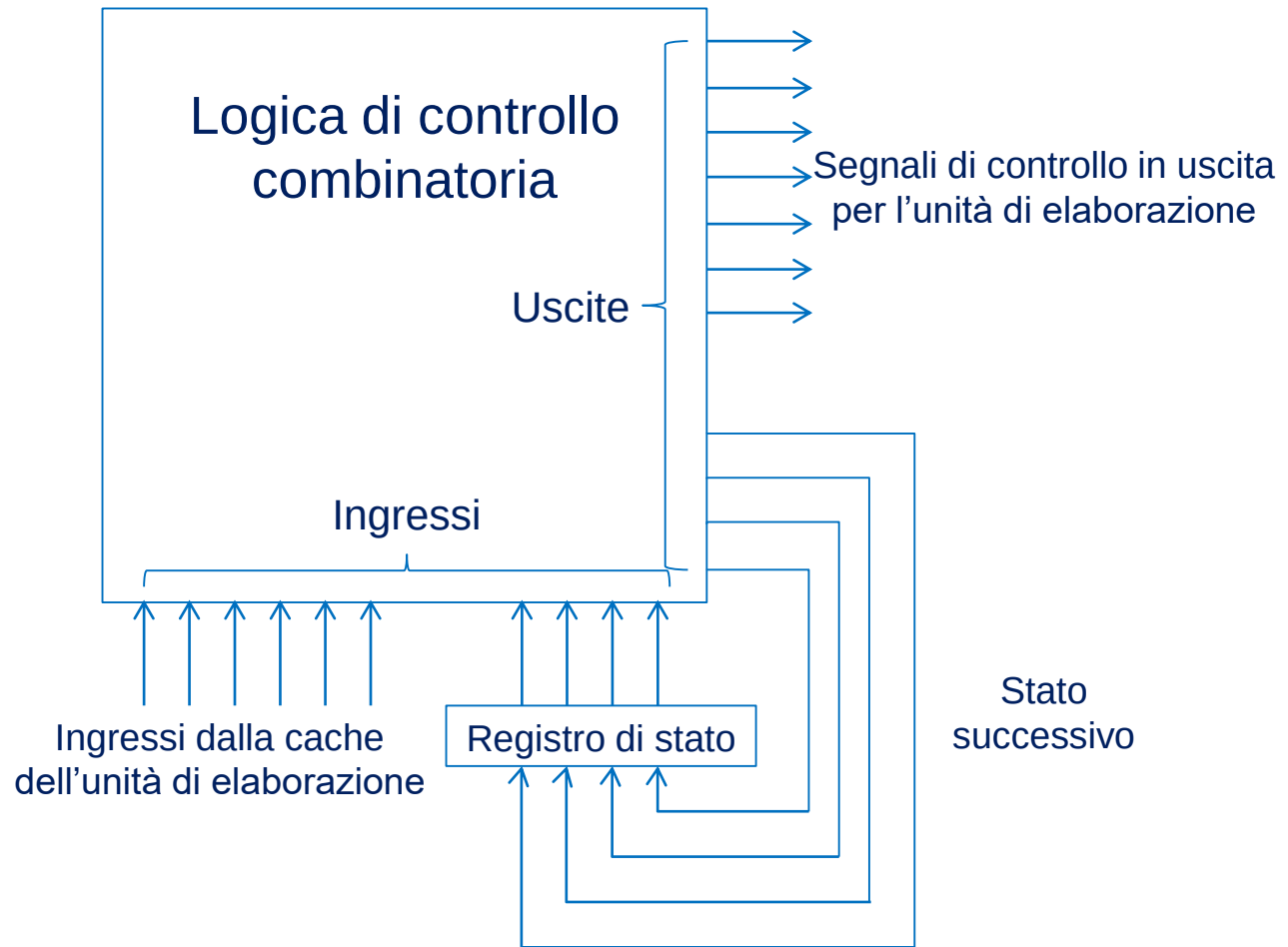
1. **Collegare** tra loro le cache in modo che ciascuna abbia nozione di cosa fanno le altre
 - Alternativamente, usare un meccanismo centralizzato di coordinamento (**directory**)
2. Permettere la **replicazione** e **migrazione** di informazioni tra le cache

Il controllore di una cache

Per progettare un controllore di cache possiamo usare un **Automa a Stati Finiti (FSA)**

Dobbiamo definire:

- Gli stati possibili
- I possibili input
- Le transizioni
- I segnali di output
- Come codificare stati e segnali



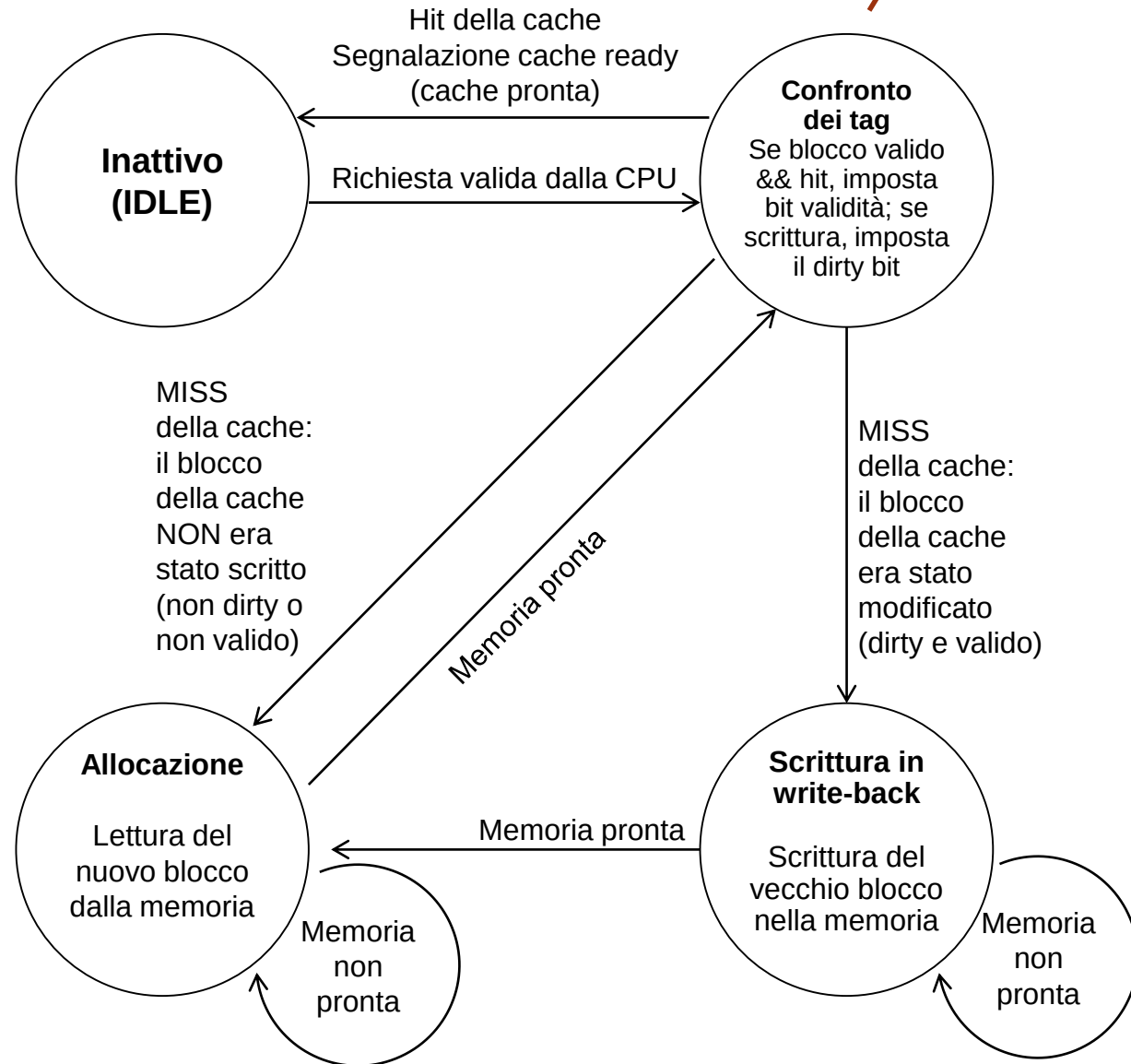
Il FSA del controllore di cache

4 stati:

- Attesa della richiesta (**idle**)
- Ricerca del blocco in cache (**confronto dei tag**)
- Lettura del blocco dalla memoria (**allocazione**)
- Scrittura del blocco in memoria (**write-back**)

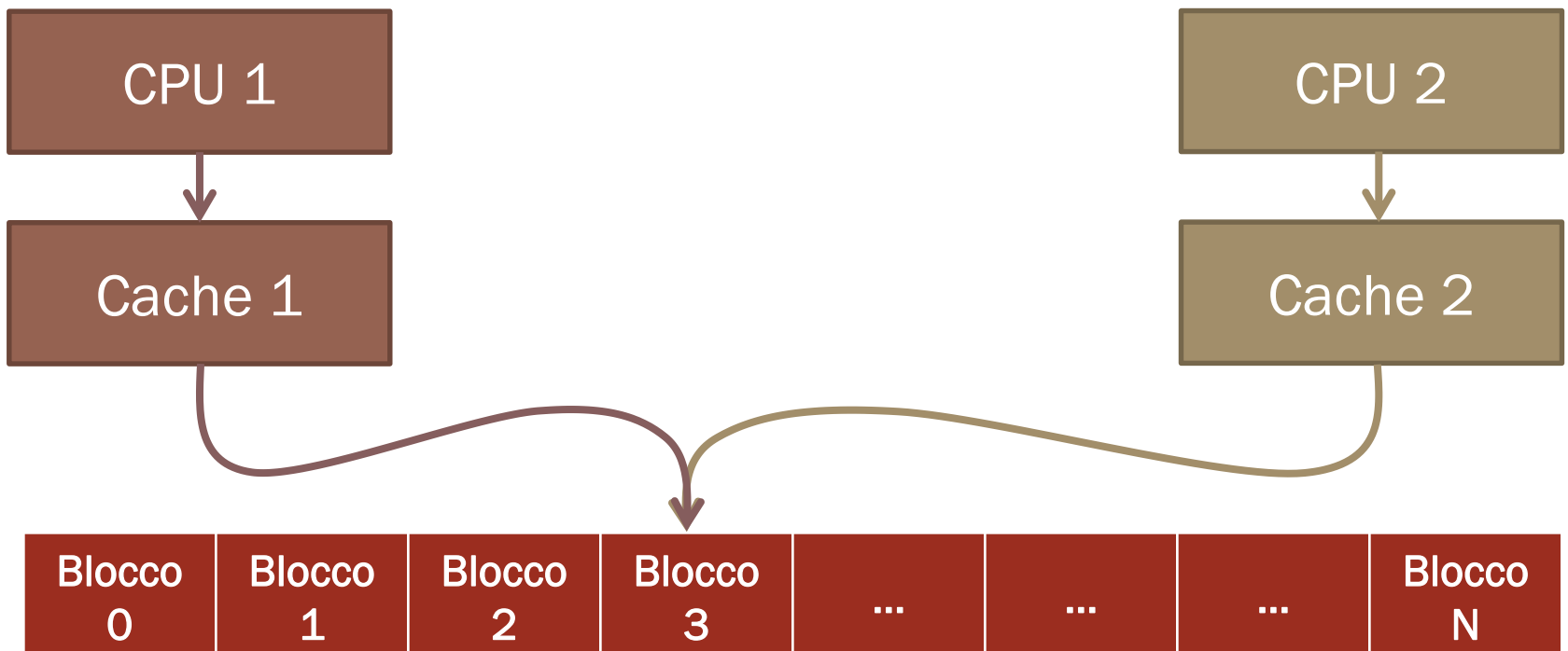
Possibili miglioramenti:

1. **Separare** confronto dei tag dalla scrittura in cache per diminuire il clock
2. **Bufferizzare** la scrittura per leggere subito il nuovo blocco e non far attendere la CPU



Parallelismo e cache multiple

- Più processori condividono la **stessa memoria fisica**
 - Ad esempio, nei sistemi multi-core
- Possono accedere contemporaneamente agli **stessi dati** tramite **cache diverse**
 - Lo **stesso blocco** è presente in **due copie diverse** in **due cache separate**



Coerenza delle cache

Informalmente: «un sistema di memoria è **coerente** se la lettura di un dato restituisce il valore che è stato scritto in quella memoria più di recente»

In realtà le cose sono più complesse:

- La **coerenza** indica quale valore deve essere restituito da una lettura
- La **consistenza** indica quando, effettuata una scrittura, il dato sarà finalmente disponibile

Più **precisamente** la coerenza è data da quanto segue:

1. Se **P1 legge** da X dopo aver scritto in X, e senza che altri scrivano in X, otterrà lo **stesso valore scritto**
 - Ci si aspetta che sia garantito in architetture mono-core
2. Se **P2 legge** da X dopo che **P1 ha scritto** in X, il **valore letto** sarà quello **scritto da P1** (se è passato abbastanza tempo e nessun altro processore ha scritto in X)
3. Le scritture nella stessa posizione X sono **serializzate**:
due scritture effettuate da processori diversi in X vengono lette da tutti i processori nello stesso ordine
4. In un programma sequenziale il «più recente» è determinato dall'ordine del programma non dal tempo

Problema di coerenza della cache

Informalmente: «un sistema di memoria è **coerente** se la lettura di un dato restituisce il valore che è stato scritto in quella memoria più di recente»

In realtà le cose sono più complesse:

- La **coerenza** indica quale valore deve essere ritornato da una lettura
- La **consistenza** indica quando, fatta una scrittura, il dato sarà finalmente disponibile

Sequenza di accessi alla stessa locazione X da due processori diversi con due cache separate

Passo	Evento	Contenuto della cache della CPU A	Contenuto della cache della CPU B	Contenuto della memoria nella locazione X
0		MISS		0
1	CPU A legge X	0	MISS	0
2	CPU B legge X	0	0	0
3	CPU A scrive 1 in X	1	0	1

The diagram illustrates a sequence of memory accesses between two processors (CPU A and CPU B) and a shared memory location X. The table shows the state of the caches and memory at each step. Red arrows and boxes highlight the flow of data and the state of the caches. In step 1, CPU A reads X (0) and CPU B has a MISS. In step 2, CPU B reads X (0) and both caches contain 0. In step 3, CPU A writes 1 to X, updating its cache to 1, while CPU B's cache remains 0. The memory location X is updated to 1.

Invarianti della coerenza della cache

Single-Writer, Multiple-Reads (SWMR): per ogni locazione di memoria x , per ogni tempo fissato (epoca) un solo processore può scrivere e leggere la locazione x , gli altri processori possono al massimo leggere il valore della locazione x

Data Value invariante (Write serialization): epoche di sola lettura, il valore contenuto dalla locazione alla fine dell'epoca è lo stesso che all'inizio dell'epoca

Implementazione della coerenza:

- **Software-based solution**
 - OS usa il page-fault per propagare le scritture
 - Funziona sui cluster
- **Hardware-based**
 - Implementazione della coerenza basata su «Snooping»
 - Implementazione della coerenza basata su «Directory»

Garantire la coerenza

In un'architettura multi-processore che sia **cache-coerente** si hanno:

- **Migrazione dei dati**
 - Quando un dato viene trasferito in una cache locale, può essere usato in modo trasparente
 - Latenza ridotta
 - Larghezza di banda ridotta
- **Replicazione dei dati**
 - Quando un dato condiviso viene letto da più processori, viene replicato nelle corrispondenti cache
 - Latenza ridotta
 - Competizione in scrittura minore

Supporto alla coerenza: **protocollo hardware di coerenza della cache**

- Lo stato di condivisione dei blocchi nelle diverse cache deve essere tracciato (meglio se da tutte le cache per usare un algoritmo distribuito)

Protocollo di snooping

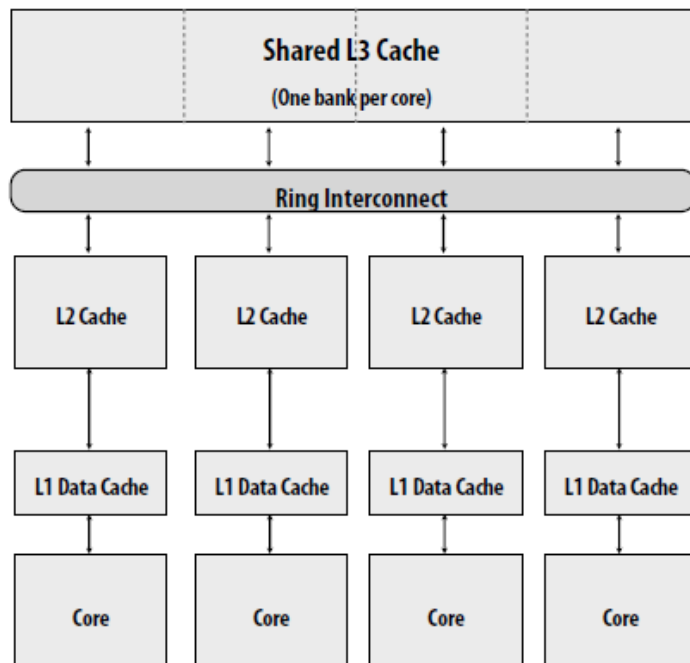
- Ogni cache che contiene un blocco ne ha anche lo stato di condivisione nelle altre cache
- Le cache sono collegate da un **bus** o una **rete** che permette a ciascuna di sbirciare («**snoop**») le altre per aggiornare lo stato di condivisione del blocco
- È sufficiente la **trasmissione delle MISS e delle scritture** da una cache a tutte le altre
 - Per far **migrare il blocco** se necessario o per **invalidare un blocco** divenuto obsoleto
 - Problema: la rete di interconnessione è difficilmente «scalabile» con meccanismi a «directory»

Esempio di **protocollo di invalidazione** con una **cache** di tipo **write-back**

Attività del processore	Attività del bus	Contenuto della cache della CPU A	Contenuto della cache della CPU B	Contenuto della memoria nella locazione X
		MISS		0
CPU A legge X	MISS cache per X	0	MISS	0
CPU B legge X	MISS cache per X	0	0	0
CPU A scrive 1 in X	Invalidazione di X	1	MISS	0
CPU B legge X	MISS cache per X	1	1	1

Protocollo basato sulle «Directories»

- L'informazione riguardante lo **stato** della linea della cache è salvato in una «**posizione**» chiamata «**Directory**»
- Le cache cercano le informazioni necessarie dalla directory
- La coerenza della cache è mantenuta tramite i messaggi **point-to-point** tra le caches in base alla necessità



- **L3 serves as centralized directory for all lines in the L3 cache**
 - **Serialization point**

(Since L3 is an inclusive cache, any line in L2 is guaranteed to also be resident in L3)

- **Directory maintains list of L2 caches containing line**
- **Instead of broadcasting coherence traffic to all L2's, only send coherence messages to L2's that contain the line**

(Core i7 interconnect is a ring, it is not a bus)

- **Directory dimensions:**
 - **P=4**
 - **M = number of L3 cache lines**

Coerenza delle cache

Informalmente: «un sistema di memoria è **coerente** se la lettura di un dato restituisce il valore che è stato scritto in quella memoria più di recente»

In realtà le cose sono più complesse:

- La **coerenza** indica quale valore deve essere ritornato da una lettura
- La **consistenza** indica quando, fatta una scrittura, il dato sarà finalmente disponibile (quindi dipende dall'ordine degli eventi osservati da ogni processore)

Circa la **consistenza**:

- Una scrittura non è considerata completa finché tutti i processori ne vedono l'effetto
- Un processore non deve cambiare l'ordine delle scritture
 - Se P1 prima scrive in X e poi in Y,
gli altri processori che leggono in Y devono poter ottenere anche il valore aggiornato in X
- I processori possono quindi riordinare le letture
ma devono completare le scritture nell'ordine giusto

$P_1:$	$W(x)1$
$P_2:$	$R(x)0 \quad R(x)1$

$P_1:$	$W(x)1$
$P_2:$	$R(x)1 \quad R(x)1$